

Standardní knihovny C

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická v Jihlavě

© Karel Richta, 2020

Základy strukturovaného programování, ZSP
10/2020, Lekce 6

<https://moodle.vspj.cz/course/view.php?id=200871>



Vysoká škola
polytechnická
Jihlava

Standardní knihovny C

<assert.h> -- pro ladění
<ctype.h> -- klasifikace tříd znaků
<errno.h> -- chybové kódy
<float.h> -- vlastnosti reprezentace v pohyblivé řádové čárce
<limits.h> -- vlastnosti reprezentace v pevné řádové čárce
<locale.h> -- lokální konvence
<math.h> -- matematické funkce
<setjmp.h> -- zpracování výjimek (dlouhé skoky)
<signal.h> -- zpracování přerušení
<stdarg.h> -- proměnný počet argumentů
<stdint.h> -- celočíselné typy
<stddef.h> -- minimální definice
<stdio.h> -- vstup a výstup
<stdlib.h> -- standardní knihovna
<string.h> -- manipulace s řetězci
<time.h> -- práce s časem

Standardní knihovny C (doplňky)

<inttypes.h> -- speciální celá čísla (velká, s danou přesností)

<complex.h> -- komplexní čísla

<iso646.h> -- makra pro logické a bitové operátory

<stdbool.h> -- makra pro logické hodnoty (bool, true, false)

<tgmath.h> -- generická makra, aby služby matematické knihovny zahrnovaly také práci s komplexními čísly

<fenv.h> -- prostředí pro práci s plovoucí řádovou čárkou (floating-point environment)

<wchar.h> -- funkce pro práci s „širokými“ znaky

<wctype.h> -- definice typů „širokých“ znaků

<uchar.h> -- definice znaků v Unicode

Ladění – `<assert.h>` (`cassert`)

- Pomoc při ladění – makro **`assert`** v hlavičkovém souboru **`<assert.h>`** (**`cassert`**)
- Makro má jako parametr podmínku:
 - je-li podmínka platná, nic se nedělá,
 - není-li podmínka platná, zobrazí pozici a podmínku, která vedla k jeho vyvolání. Zároveň ukončí program.
- Volání **`assert`** se typicky nechají v kódu:
 - pro ladění se použijí,
 - ve finální verzi se hromadně odpojí definicí:

```
#define NDEBUG
```

Příklad: <assert.h> (cassert)

```
#include <stdio.h>      /* printf */
#include <assert.h>      /* assert */
void print_number(int* myInt) {
    assert (myInt!=NULL);
    printf ("%d\n",*myInt);
};

int main() {
    int a = 10;
    int *b = &a;
    int *c = NULL;
    print_number(b);
    print_number(c);
};
```



Assertion failed: myInt != NULL, file
d:\výuka\fel\a7b36pjc\prednasky\2015\priklady\assert\assert\assert.cpp, line 12
abnormal program termination

Kategorie znaků: <ctype.h> (ctype)

Klasifikační funkce:

- **isalnum** – test na alfanumerický znak
- **isalpha** – test na alfabetycký znak
- **isblank** – test na “bílý znak” (mezera, tabulátor)
- **iscntrl** – test na řídicí znak
- **isdigit** – test na číslici
- **isgraph** – test zda má znak grafickou reprezentaci
- **islower** – test na malá písmena
- **isprint** – test na tištitelný znak
- **ispunct** – test na oddělovač (punctuation character)
- **isspace** – test na tzv. “bílý znak” (mezera, tabulátor)
- **isupper** – test na velká písmena
- **isxdigit** – test na hexadecimální cifry

Konverzní funkce:

- **tolower** – konverze na malá písmena
- **toupper** – konverze na velká písmena

Příklad: <ctype.h> (ctype)

```
/* ctype example */
#include <stdio.h>
#include <ctype.h>
int main () {
    char str[]="c3F-.";
    for (auto i = 0; i<5; i++) {
        if (isalnum(str[i])) printf("Znak %c je alfanumericky\n",str[i]);
        if (isalpha(str[i])) printf("Znak %c je alfabeticky\n",str[i]);
        if (isdigit(str[i])) printf("Znak %c je cifra\n",str[i]);
        if (isxdigit(str[i])) printf("Znak %c je hexadecimalni cifra\n",
                                    str[i]);
        if (ispunct(str[i])) printf("Znak %c je oddelovac\n",str[i]);
    };
    return 0;
}
```

Znak c je alfanumericky
Znak c je alfabeticky
Znak c je hexadecimalni cifra
Znak 3 je alfanumericky
Znak 3 je cifra
Znak 3 je hexadecimalni cifra
Znak F je alfanumericky
Znak F je alfabeticky
Znak F je hexadecimalni cifra
Znak - je oddelovac
Znak . je oddelovac

Chybové stavy: `<errno.h>` (`cerrno`)

- Volání knihovní funkce nemusí být úspěšné. Většina funkcí je navržena tak, aby se z návratové hodnoty poznalo, že došlo k nějaké chybě.
- Pokud funkce vrací celé číslo (*int*), hodnota 0 obecně znamená úspěch, jiné hodnoty chybu. Pokud funkce vrací ukazatel, hodnota **NULL** signalizuje chybu.
- Podrobnější informace lze získat z proměnné (norma C připouští i implementaci pomocí makra, které se chová jako proměnná) **errno** z hlavičkového souboru **errno.h**.
- Na počátku chodu programu je **errno** 0, ale neúspěšné volání (téměř) každé funkce ze standardní knihovny hodnotu **errno** nastaví na charakteristickou hodnotu.

Chybové stavy: `<errno.h>` (`cerrno`) (pokr.)

- Pokud knihovní funkce neuspěje a nastaví ***errno***, volajícího zpravidla víc než kód chyby zajímá její popis. V ***errno.h*** jsou kromě ***errno*** deklarované také:

`const char *sys_errlist[]; int sys_nerr;`

- Tedy pole indexovatelné hodnotami ***errno*** až do ***sys_nerr - 1***. Hlídat rozsah pole ***sys_errlist*** při každém ošetření chyby je jistě otravné, nehlídat ho zase nebezpečné, neboť v případě nekompatibility mezi verzemi knihoven se může ***errno*** dostat mimo rozsah pole. Lepší je použít funkci ***strerror*** ze ***string.h*** nebo v případě prostého výpisu na chybový výstup použít ***perror*** ze ***stdio.h***.

Příklad: <errno.h> (cerrno)

```
#include <errno.h> // errno
#include <string.h> // strerror
#include <stdio.h> // printf, FILE, fopen

int main() {
    printf("Pokus o otevreni souboru\n");
    FILE *f = fopen("/proc/cpuinfo", "w");
    if (f == NULL) {
        printf("Neco je spatne! Chyba: %d: %s\n",
               errno, strerror(errno));
    }
    return 0;
}
```

Pokus o otevreni souboru
Neco je spatne! Chyba: 2: No such file or directory

Charakteristiky pevné čárky: `<limits.h>` (`climits`)

- `CHAR_BIT` počet bitů objektů typu `char`
- `CHAR_MIN` minimální hodnota `char`
- `CHAR_MAX` maximální hodnota `char`
- `MB_LEN_MAX` maximální počet bytů pro více-bytové znaky
- `SHRT_MIN` minimální hodnota `short int`
- `SHRT_MAX` maximální hodnota `short int`
- `INT_MIN` minimální hodnota `int`
- `INT_MAX` maximální hodnota `int`
- `LONG_MIN` minimální hodnota `long int`
- `LONG_MAX` maximální hodnota `long int`

Charakteristiky float: <float.h> (cfloat)

Charakteristiky zobrazení v pohyblivé řádové čárce:

hodnota v pohyblivé řádové čárce = mantisa x základ^{exponent}

Sada maker předznačených FLT,DBL,LDBL:

- **FLT_RADIX** – základ pro všechny typy (float, double and long double)
- **FLT_MANT_DIG, DBL_MANT_DIG, LDBL_MANT_DIG**
 - počet platných míst mantisy
- **FLT_MIN_EXP, DBL_MIN_EXP, LDBL_MIN_EXP**
 - minimální počet platných míst exponentu
- **FLT_MAX_EXP, DBL_MAX_EXP, LDBL_MAX_EXP**
 - minimální počet platných míst exponentu

Charakteristiky (pokr.): <float.h> (cfloat)

- **FLT_MAX, DBL_MAX, LDBL_MAX**
 - maximální zobrazitelné číslo
- **FLT_MIN, DBL_MIN, LDBL_MIN**
 - minimální zobrazitelné číslo
- **FLT_EPSILON, DBL_EPSILON, LDBL_EPSILON**
 - rozdíl mezi 1 a minimálním zobrazitelným číslem větším než 1
- **FLT_ROUNDS**
 - chování při zaokrouhlení (ROUND), možné hodnoty jsou:
 - 1 nedefinováno
 - 0 směrem k nule
 - 1 směrem k nejbližšímu
 - 2 směrem ke kladnému nekonečnu
 - 3 směrem k zápornému nekonečnu

Příklad: <float.h> (cfloat)

```
#include <float.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    printf("Maximalni float: %f\n" , FLT_MAX);
    printf("Maximalni double: %f\n" , DBL_MAX);
    printf("Maximalni long double: %Lf\n" , LDBL_MAX);
    printf("Minimalni float: %f\n" , FLT_MIN);
    printf("Minimalni double: %f\n" , DBL_MIN);
    printf("Minimalni long double: %Lf\n" , LDBL_MIN);
    printf("Zaokrouhleni: %f\n" , FLT_ROUNDS);
    system("pause");
    return 0;
}
```

Charakteristiky lokality: <locale.h> (clocale)

- Deklarace struktury lconv, obsahující charakteristiky lokality
- Funkce

struct lconv* localeconv (void);

- získání charakteristik

char* setlocale (int category, const char* locale);

- nastavení charakteristik

Příklad: <locale.h> (clocale)

```
#include <stdio.h>      /* printf */
#include <locale.h>      /* setlocale, LC_MONETARY,
                        struct lconv, localeconv */

int main () {
    setlocale (LC_MONETARY, "");
    struct lconv * lc;
    lc=localeconv();
    printf("Local Currency Symbol: %s\n" ,
           lc->currency_symbol);
    printf("International Currency Symbol: %s\n" ,
           lc->int_curr_symbol);
    return 0;
}
```


Alokace paměti: <stdlib.h> (cstdlib)

void *malloc(size_t Delka);

- alokace paměti zadané Delky

void *calloc(size_t Pocet, size_t Delka);

- alokace paměti pro Pocet položek uvedené Delky; pamť je vynulována

void *realloc(void *Ukaz, size_t NovaDelka);

- zvětší či zmenší dříve přidělenou paměť (Ukaz)
- při zvětšení může dojít k přesunu na jinou adresu; v tom případě se přesune i obsah!
- je-li Ukaz == NULL, pak funguje stejně jako malloc

void free(void *Ukaz);

- uvolní paměť dříve alokovanou funkcemi malloc, calloc nebo realloc
- pro Ukaz == NULL nedělá nic

Textové konverze: <stdlib.h> (cstdlib)

int atoi(char *Text); dekadická konverze na int

- = (int)strtol(Text, (char **)NULL, 10)

long atol(char *Text); dekadická konverze na long

- = strtol(Text, (char **)NULL, 10)

long strtol(char *Text; char **Konec; int Zaklad);

- konverze text --> celé číslo;
obecná číselná soustava: základ 2 - 36
- může obsahovat úvodní mezery, tabulátory, znaménko a posloupnost číslic v uvedené soustavě
- je-li Zaklad == 0, pak akceptuje a rozlišuje čísla ve tvaru lexikálních symbolů (0x - hexa, 0 - oktal, jinak dekadické); nepřijímá příponu L resp. U
- přes parametr Konec funkce vrátí ukazatel na první znak, který již není součástí zápisu čísla; je-li Konec == NULL, pak se nepřisazuje

Textové konverze (pokračování)

`unsigned long strtoul(char *Text; char **Konec; int Zaklad);`

- totéž jako `strtol`, jen vrací `unsigned long`

`double atof(char *Text);` konverze na `double`

- = `strtod(Text, (char **)NULL)`

`double strtod(char *Text; char **Konec);`

- viz `strtol`; zápis je možný jen dekadicky

Příklady:

```
atoi(" -123abc") == -123
```

```
atol("ASD") == 0 /* Chyba! Nepozná se! */
```

```
strtol(" 0xABCG0",&Ptr,0) == 0x00000ABC; Ptr == "G0"
```

```
strtoul("1111",NULL,2) == 15
```

```
atof(" 1.1233e12e12") == 1.1233E12
```

```
strtod(" -123E14",&Ptr) == -123.0E14; Ptr == ""
```

Komunikace s prostředím: `<stdlib.h>` (`cstdlib`)

`void abort(void);`

- předčasné ukončení programu, návrat do systému
- nemusí uzavřít soubory!

`void exit(int Stav);`

- ukončení programu, návrat do systému
- vyrovnávací paměti jsou zapsány, soubory uzavřeny
- Stav je předán systému jako výsledek programu

`int atexit(void (*Funct)(void));`

- instaluje funkci, která se volá při korektním ukončení programu (exit, return ve funkci main)
- funkce se volají v opačném pořadí, než v jakém se instalovaly
- takto je možno instalovat přinejmenším 32 funkcí

`char *getenv(char *Jmeno);`

- vrací hodnotu systémové proměnné daného Jména (např. PATH)

`int system(char *Prikaz);`

- umožňuje vyvolat Příkaz operačního systému

Různé funkce: <stdlib.h> (cstdlib)

int rand(void);

- generátor pseudo-náhodných čísel, vrací číslo z intervalu 0 - RAND_MAX (min. 32767)

void srand(unsigned Seed);

- inicializuje generátor pseudo-náhodných čísel

**void *bsearch(void *Klic, void *Tabulka, size_t PocetPolozek,
size_t DelkaPolozky,
int (*Porovnej)(void *Klic, void *Polozka));**

- algoritmus hledání půlením v uspořádané Tabulce
- funkce Porovnej musí vracet číslo 0, je-li Klíč roven klíči hledané položky; číslo < 0, je-li hledaný klíč menší; jinak číslo > 0
- vrací adresu nalezené položky resp. NULL, nebyla-li položka nalezena

**void qsort(void *Pole, size_t PocetPolozek, size_t DelkaPolozky,
int (*Porovnej)(void *Prvni, void *Druhy));**

- algoritmus řazení Pole (quick sort); uspořádá Pole vzestupně
- porovnávací funkce viz bsearch

int abs(int Cislo); long labs(long Cislo);

- absolutní hodnota celého Čísla

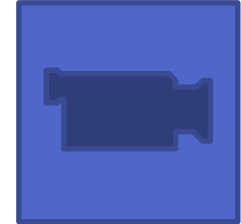
Příklad: QuickSort

```
#include <stdio.h>      /* printf */
#include <stdlib.h>      /* qsort */

int values[] = { 40, 10, 100, 90, 20, 25 };

int compare (const void * a, const void * b) {
    return ( *(int*)a - *(int*)b );
}

int main () {
    int n;
    qsort (values, 6, sizeof(int), compare);
    for (n=0; n<6; n++) printf ("%d ", values[n]);
    return 0;
}
```



Řetězcové funkce: <string.h> (cstring)

size_t strlen(char *Retezec);

- vrátí délku Řetězce

char *strcpy(char *Cil, char *Zdroj);

- kopírování řetězců; za alokaci dostatku paměti pro Cíl je zodpovědná volající funkce

char *strncpy(char *Cil, char *Zdroj, size_t MaxDelka);

- kopírování nejvýše MaxDelka znaků; výsledek nemusí být korektně zakončený řetězec!

char *strcat(char *Cil, char *Zdroj);

- spojování řetězců; za alokaci dostatku paměti pro Cíl je zodpovědná volající funkce

char *strncat(char *Cil, char *Zdroj, size_t MaxDelka);

- kopírování nejvýše MaxDelka znaků Zdrojového řetězce na konec Cílového; výsledek je vždy korektní zakončený řetězec

Řetězcové funkce (pokračování)

int strcmp(char *Prvni, char *Druhy);

- porovnání dvou řetězců

int strncmp(char *Prvni, char *Druhy, size_t N);

- porovnání nejvýše prvních N znaků řetězců

char *strchr(char *Retezec, int Znak);

- hledání prvního výskytu Znaku v Řetězci

char *strrchr(char *Retezec, int Znak);

- hledání posledního výskytu Znaku v Řetězci

char *strstr(char *Retezec, char *Podretezec);

- hledání prvního výskytu Podřetězce v Řetězci
- vrátí adresu prvního znaku Podřetězce v Řetězci resp. NULL

Blokové funkce: <string.h> (cstring)

void *memcpy(void *Cil, void *Zdroj, size_t Delka);

- kopíruje úsek paměti; vrací adresu Cílového úseku; neřeší překrývání

void *memmove(void *Cil, void *Zdroj, size_t Delka);

- jako memcpy, ale funguje i pro překrývající se úseky

int memcmp(void *První, void *Druhý, size_t Delka);

- porovnání stejnohlých bytů (interpretovaných jako unsigned char) úseků paměti
- vrací 0, jsou-li úseky shodné; vrací číslo < 0, je-li První < Druhý; jinak vrací číslo > 0

void *memchr(void *Adresa, int Byte, size_t Delka);

- hledání prvního výskytu Byte (interpretovaný jako unsigned char) v úseku paměti daném Adresou a Délkou
- vrátí adresu Byte v úseku paměti resp. NULL

void *memset(void *Adresa, int Byte, size_t Delka);

- vyplní úsek paměti daný Adresou a Délkou daným Bytem (interpretovaným jako unsigned char)

Matematické funkce: <math.h> (cmath)

<code>double sin(double X);</code>	<code>sin X</code>
<code>double cos(double X);</code>	<code>cos X</code>
<code>double tan(double X);</code>	<code>tg X</code>
<code>double asin(double X);</code>	<code>arcsin X</code>
<code>double acos(double X);</code>	<code>arccos X</code>
<code>double atan(double X);</code>	<code>arctg X</code>
<code>double atan2(double X, double Y);</code>	<code>arctg (X/Y)</code>
<code>double sinh(double X);</code>	<code>sinh X</code>
<code>double cosh(double X);</code>	<code>cosh X</code>
<code>double tanh(double X);</code>	<code>tgh X</code>
<code>double exp(double X);</code>	e^X
<code>double log(double X);</code>	$\ln X$
<code>double log10(double X);</code>	$\log_{10} X$
<code>double pow(double X, double Y);</code>	X^Y
<code>double sqrt(double X);</code>	$X^{1/2}$
<code>double fabs(double X);</code>	$ X $
<code>double ceil(double X);</code>	horní celá část X
<code>double floor(double X);</code>	dolní celá část X

Příklad – sinusovka

```
// (C) Sallyx.org
#include <stdio.h>
#include <stdlib.h>
#define _USE_MATH_DEFINES /* Kvuli Visual Studio */
#include <math.h>
int main(void)
{
    int ia;
    float fa;
    /* kresleni sinusove vlny */
    for (fa = 0.0F; fa < 2.0F * M_PI; fa += (float)M_PI / 13) {
        ia = 40 + (int)(20.0 * sin(fa));
        printf("%*c\n", ia, '*');
    }
    system("Pause");
    return 0;
}
```



Práce s časem: `<time.h>` (`ctime`)

Uchovávání času

- Aby bylo možné čas uchovávat v co možná nejmenším počtu bitů, zaznamenává se jako počet sekund od určitého data. Například v Unixových systémech je to od 1. 1. 1970, v MS-DOSu je to 1. 1. 1980 atd. V dnešní době je to víceméně zátěž z historických důvodů.
- S časem uloženým v sekundách se pracuje pomocí předdefinovaného datového typu **`time_t`**. Je to celočíselný typ.
- Pokud v programu zaznamenáte dvě takovéto čísla a odečtete je od sebe, zjistíte, kolik sekund mezi jejich zaznamenáním uběhlo. Takovéto jednoduché odečtení však není dle normy ANSI, proto používejte funkci `difftime()`, která je k tomu určena (kvůli přenositelnosti na systémy, kde **`time_t`** není definován jako celé číslo).

Práce s časem – pokr.

- Rozumněji se pracuje s předdefinovanou strukturou – **tm**. Její definice vypadá takto:

```
struct tm {  
    int tm_sec; /* vteřiny 0-60 */  
    int tm_min; /* minuty 0-59 */  
    int tm_hour; /* hodiny 0-23 */  
    int tm_mday; /* dny v měsíci 1-31 */  
    int tm_mon; /* měsíc 0-11 */  
    int tm_year; /* rok od 1900 */  
    int tm_wday; /* den v týdnu 0-6, 0 je nedele */  
    int tm_yday; /* den v roce 0-365 */  
    int tm_isdst; /* příznak letního času*/  
};
```

Výčet funkcí: <time.h> (ctime)

char *asctime(const struct tm *timeptr);

Převeď strukturu *tm* na řetězec ("Wed Jun 30 21:49:08 1993\n"). Návratovou hodnotou je ukazatel na tento řetězec.

char *ctime(const time_t *timep);

Vrací to samé, jako `asctime(localtime(timep))`;

double difftime(time_t time1, time_t time0);

Vrací rozdíl mezi časem *time0* a *time1* ve vteřinách.

struct tm *gmtime(const time_t *timep);

Převeď čas uložený v *timep* (pomocí vteřin) na strukturu *tm* a to v čase UTC (GMT). Vrací *ukazatel* na tuto strukturu.

struct tm *localtime(const time_t *timep);

Jako funkce `gmtime()`, ale vrací lokální čas (nikoliv UTC).

Výčet funkcí: <time.h> (ctime)

time_t mktime(struct tm *timeptr);

Převádí strukturu na kterou odkazuje timeptr do počtu vteřin, jež jsou návratovou hodnotou.

time_t time(time_t *t);

Vrací počet sekund reprezentující čas (v Unixech počet sekund od 1.1.1970). Tuto hodnotu také uloží tam, kam ukazuje ukazatel t, pokud nemá hodnotu NULL).

Všimněte si, že je to jediná funkce, která vám vrátí aktuální čas.

size_t strftime(char *s, size_t max, const char *format, const struct tm *tm);

Tato funkce převádí strukturu *tm* na řetězec. Tento řetězec bude uložen do pole, na které ukazuje ukazatel *s*. Maximálně však *max* znaků. Co bude řetězec obsahovat, to je dáno formátovacím řetězcem *format*. Co může tento formátovací řetězec obsahovat najdete v tabulce níže.

Příklad: <time.h> (ctime)



```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <time.h>
#define MAX 80

int main(void) {
    time_t cas;
    struct tm gtmcas;
    struct tm *ltmcas;
    char retezec[MAX];
    char *uch;

    cas = time(NULL);

    gtmcas = *(gmtime(&cas));
    ltmcas = localtime(&cas);
```


Příklad (pokr.)

```
/* vsimnete si, ze retezec vraceny funkcemi ctime a asctime je jednak
 * v anglictine a take na konci ukoncen znakem '\n'. Texty LC a UTC kvuli
 * tomu prejdu az na dalsi radek */
printf("Je prave \"%s\" LC\n", ctime(&cas));
printf("Je prave \"%s\" GMT\n", asctime(&gmtcas));
printf("Je prave \"%s\" LC\n", asctime(ltmcas));

strftime(retezec, MAX - 1, "Je prave %A %d.%m.%Y, %I:%M %p LC",
        ltmcas);
printf("%s\n", retezec);
printf("Tm(LC): %02i.%02i.%04i\n", ltmcas->tm_mday,
        ltmcas->tm_mon, ltmcas->tm_year + 1900);
cas = (time_t) 0;
printf("Pocatek vseho je %s", ctime(&cas));

return 0;
}
```

Prostředí pro plovoucí čárku: `fenv.h` (`cfenv`)

- Typ `fenv_t` – struktura zahrnující veškerý stav prostředí pro plovoucí čárku (implementačně závislé), včetně příznaků a výjimek.

- Funkce:

```
int fesetenv(const fenv_t* envp); // nastavení prostředí
int fegetenv(fenv_t* envp);      // získání stavu prostředí
int feupdateenv(const fenv_t* envp); // úprava prostředí
int feclearexcept(int excepts); // vymaže výjimky
int feraiseexcept(int excepts); // generuje výjimku
int fegetexceptflag(fexcept_t* flagp, int excepts);
int fesetexceptflag(const fexcept_t* flagp, int excepts);
int feholdexcept(fenv_t* envp); // schová výjimky a maskuje přer.
```

- Implicitní nastavení prostředí (jako při startu):

```
FE_DFL_ENV
```

- Pragma (umožní používání `fenv`, pokud existuje):

```
FENV_ACCESS
```

Příklad: fenv.h (cfenv)

```
#include <stdio.h>      /* printf, puts */
#include <fenv.h>        /* feholdexcept, feclearexcept, fetestexcept, feupdateenv, FE_* */
#include <math.h>        /* log */
#pragma STDC FENV_ACCESS on

double log_zerook(double x) {
    fenv_t fe;
    feholdexcept(&fe);
    x=log(x);
    feclearexcept(FE_OVERFLOW|FE_DIVBYZERO);
    feupdateenv(&fe);
    return x;
}
```

```
int main ()
{
    feclearexcept(FE_ALL_EXCEPT);
    printf ("log(0.0): %f\n", log_zerook(0.0));
    if (!fetestexcept(FE_ALL_EXCEPT))
        puts ("no exceptions raised");
    return 0;
}
```

log(0.0): -inf
no exceptions raised

Dlouhé skoky: `stdjmp.h` (`cstdjmp`)

Poskytuje služby:

- Definici typu (struktura pro uložení aktuálního kontextu):

`jmp_buf`

- Funkci:

```
void longjmp(jmp_buf env, int val); /* dlouhý skok do  
kontextu env, val je navratová hodnota */
```

- Makro:

```
int setjmp(jmp_buf env); /* záznam kontextu do env, vrátí  
hodnotu 0, když je volána přímo, nebo hodnotu val, pokud je  
vyvolána přes longjmp */
```

Příklad

```
/* longjmp example */
#include <stdio.h>      /* printf */
#include <setjmp.h>      /* jmp_buf, setjmp, longjmp */

main()
{
    jmp_buf env;
    int val;

    val=setjmp(env);

    printf ("val is %d\n",val);

    if (!val) longjmp(env, 1);

    return 0;
}
```

Výstup:
Val is 0
Val is 1

Funkce s proměnným počtem parametrů: `stdarg.h` (`cstdarg`)

- Definuje typ: `va_list`

- Funkce (makra):

```
void va_start (va_list ap, paramN);
```

```
void va_end (va_list ap);
```

```
type va_arg (va_list ap, type);
```

```
#include <stdarg.h>
```

```
int K(int, ...); /* proměnný počet parametrů */
```

```
int printf(char *format, ...);
```

Příklad: definice funkce s proměnným počtem parametrů: **stdarg.h**

- Hlavička: paměťová třída, typ návratové hodnoty, jméno funkce a seznam deklarací pevných parametrů zakončený ... (ellipsis):

```
#include <stdarg.h>
long Sum(int n, ... ) {
    long result = 0; va_list pt;
    va_start(pt, n);
    while (n-- >= 0) result += va_arg(pt, int);
    va_end(pt);
    return result;
}
long l = Sum(4, 2, 45, 3, 17);
```

- Při volání funkcí s proměnným počtem parametrů se kontrolují pevné parametry, proměnné parametry se samozřejmě kontrolovat nedají.

Zpracování přerušení: `signal.h` (`csignal`)

- Funkce:

```
void (*signal(int sig, void (*func)(int)))(int); /*  
nastavení ovladače pro přerušení sig */  
int raise (int sig); /* generování přerušení sig */
```

- Makra

SIGABRT	Striktní ukončení (Signal Abort) – jako by byla volána funkce abort .
SIGFPE	Chyba při aritmetické operaci (např. dělení nulou).
SIGILL	Ilegální instrukce (Signal Illegal Instruction), zpravidla pokus interpretovat data.
SIGINT	Signál přerušení uživatelem (Signal Interrupt).
SIGSEGV	Nedovolený přístup do paměti (Signal Segmentation Violation).
SIGTERM	Signál pro ukončení programu (Signal Terminate).

Standardní knihovny C++

- **Algoritmy** <algorithm> - funkce standardní knihovny šablon (STL), které provádějí algoritmy.
- **Knihovny odpovídající standardním knihovnám C** - <cassert> , <cctype>, <cerrno>, <cfenv>, <cfloat>, < cinttypes>, <ciso646>, <climits>, <locale>, <cmath>, <csetjmp>, <csignal>, <cstdarg>, <cstdbool>, <cstddef>, <cstdint>, <cstdio>, <cstdlib>, <cstring>, <ctgmth>, <ctime>, <wchar>, <cwctype>
- **Kontejnery** - <array> , <deque>, <forward list>, <list>,<vector>, <map> , <set>, <unordered map> , <unordered set>, <queue> , <stack> - funkce standardní knihovny šablon (STL), které pracují s kontejnery.
- **Zpracování chyb** - <exception>, <stdexcept>,<system error>
- **Vstup/Výstup** - <filesystem> , <fstream>, <iomanip>, <ios>, <iosfwd>, <iostream>, <istream>, <ostream>, <sstream>, <streambuf>, <strstream>
- **Iterátory** - <iterator>

Standardní knihovny C++

- **Lokalizace** - <codecvt> , <cvt/wbuffer>, <cvt/wstring>, <locale>
- **Matematické knihovny** - <complex> , <limits>, <numeric>, <random>, <ratio>, <valarray>
- **Správa paměti** - <allocators> , <memory>, <new>, <scoped_allocator>
- **Podpora paralelismu** - <atomic> , <condition_variable>, <future>, <mutex>, <thread>
- **Řetězce a znaková data** - <regex> , <string>
- **Ostatní** - <bitset> , <chrono>, <functional>, <initializer_list>, <tuple>, <type_traits>, <typeinfo>, <typeindex>, <utility>

The End